Лекція №20 (2 години). Стилізація у React: Styled Components та Tailwind CSS.

План лекції

1. Еволюція CSS: від глобальних стилів до модульності.
2. Традиційні підходи: глобальний CSS, Inline-стилі та CSS Modules.
3. CSS-in-JS: філософія інкапсуляції стилів.
4. Знайомство з `styled-components`: синтаксис Tagged Template Literals.
5. Передача Props у Styled Components для динамічної стилізації.
6. Utility-first CSS: філософія Tailwind CSS.
7. Базові класи Tailwind (Layout, Spacing, Typography, Colors).
8. Порівняння підходів: коли що обрати.

Перелік умовних скорочень

- **CSS** (Cascading Style Sheets) — каскадні таблиці стилів.
- **BEM** (Block Element Modifier) — методологія іменування CSS-класів.
- **CSS-in-JS** — підхід, при якому CSS-код пишеться безпосередньо в JavaScript-файлах.
- **Utility-first CSS** — підхід до стилізації на основі атомарних класів.

Вступ

Створивши структуру React-додатка за допомогою компонентів, ми стикаємося з наступним викликом: як усе це красиво оформити? Історично склалося так, що CSS був розроблений для документів, де стилі застосовуються глобально (до всіх тегів `p`, до всіх класів `.button`). Але в компонентній архітектурі React ми хочемо, щоб стилі кнопки `<SidebarButton>` впливали **тільки** на неї і не зламали випадково кнопку `<SubmitButton>` у формі коментарів через конфлікт імен блоків чи їх вкладеності.

Спроби вирішити цю проблему (ізоляцію) призвели до появи кількох абсолютно різних підходів до стилізації у React екосистемі: від суворої ізоляції CSS Modules до радикального CSS-in-JS та атомарного Tailwind CSS.

Мета цієї лекції — розібрати сучасні стандарти стилізації React-додатків, спробувати написати компоненти за допомогою `styled-components` та відчути швидкість верстки з `Tailwind CSS`.

1. Еволюція CSS: від глобальних стилів до модульності

Проблема глобального CSS: У класичній веб-розробці всі стилі лежать в одному величезному `style.css` (або скомпільованому з SASS). Коли проект росте, виникають конфлікти імен:

- Розробник `A` створив клас `.card` для відгуків.
- Розробник `B` (на іншій сторінці) теж створив клас `.card` для списку товарів.
- В результаті один стиль перезаписує інший, порушуючи верстку (CSS Specificity Wars).

Щоб цього уникнути, придумали жорсткі правила іменування — **БЕМ (BEM)**: `.card__title--active`. Але писати і підтримувати такі довгі класи важко.

Тому індустрія почала шукати способи **інкапсулювати** (ізолювати) CSS на рівні окремих React-компонентів.

2. Традиційні підходи у React

До того як перейти до сучасних лідерів, згадаємо три вбудовані способи стилізації:

1. Глобальні таблиці стилів (Звичайний CSS) Просто імпортуємо файл `import './App.css';` у компонент. *Мінус:* Стилі все одно стають глобальними для всієї сторінки. Конфлікти неминучі.

2. Inline-стилі (Вбудовані) Стилі передаються безпосередньо в атрибут `style` як JS-об'єкт.

```
// Зверніть увагу: camelCase і об'єкт замість рядка
<button style={{ backgroundColor: "red", marginTop: "10px" }}>Клікни</button>
```

Мінус: Неможливо використовувати Media Queries (`@media`), псевдокласи (`:hover` , `:active`) та ключові (keyframes) анімації.

3. CSS Modules Це вбудований у Vite/CRA механізм. Ви створюєте файл з назвою `Button.module.css`. При його імпорті всі імена класів автоматично перетворюються на унікальні хеші (наприклад, `.btn_xz89a`), тому конфлікти виключені.

```
import styles from "./Button.module.css";
// ...
<button className={styles.submitBtn}>Відправити</button>;
```

Мінус: Це все ще окремі CSS-файли, що не мають прямого доступу до JS-змінних компонента (State/Props).

3. CSS-in-JS: Філософія інкапсуляції

Що, якби ми взагалі перестали писати окремі файли `.css`? **CSS-in-JS** — це парадигма, де CSS-код пишеться з використанням JavaScript безпосередньо у файлі компонента. Найпопулярніша бібліотека, що реалізує цей підхід — `styled-components`.

Головна її ідея полягає в тому, що замість написання CSS-класів і подальшого призначення їх HTML-тегам, ми створюємо **нові готові React-компоненти**, у які вже намертво вшиті потрібні нам стилі.

4. Знайомство з `styled-components`

Встановлення бібліотеки:

```
npm install styled-components
```

Синтаксис ґрунтується на унікальній фішці JavaScript ES6 — **Tagged Template Literals** (тег-шаблони рядків з використанням зворотних апострофів `` ` ``).

```
import styled from "styled-components";

// 1. Створюємо стилізований компонент (завжди з великої літери!)
// styled.button означає, що під капотом це буде звичайний HTML-тег <button>
const StyledButton = styled.button`
  background-color: blue;
  color: white;
  padding: 10px 20px;
  border-radius: 5px;
  border: none;
  cursor: pointer;

  /* Ми можемо використовувати звичайний CSS синтаксис, навіть псевдокласи! */
  &:hover {
    background-color: darkblue;
  }
`;

function App() {
  return (
    <div>
      {/* 2. Просто використовуємо його як звичайний React компонент */}
```

```
      <StyledButton>Натисни мене</StyledButton>
    </div>
  );
}
```

Під капотом бібліотека сама згенерує випадковий, унікальний клас (наприклад, `.sc-aVfwe`) і вставить його у `<head>` сторінки. Конфлікти виключені на 100%.

5. Передача Props у Styled Components

Наймогутніша фішка `styled-components` — це здатність динамічно змінювати CSS на основі Пропсів (Props) вашого React компонента.

Оскільки наші стилі написані всередині JS-шаблону (в апострофах ` `), ми можемо вставляти туди JS-функції за допомогою конструкції `${...}`. Ця функція першим аргументом отримає всі пропси компонента.

```
import styled from "styled-components";

// Якщо пропс primary = true, колір буде синім, інакше - сірим
const ActionButton = styled.button`
  padding: 10px 20px;
  border-radius: 4px;
  color: white;

  /* Динамічний блок: */
  background-color: ${(props) => (props.primary ? "blue" : "gray")};

  /* Ще один приклад складнішої логіки (розмір) */
  font-size: ${(props) => (props.size === "large" ? "24px" : "16px")};
`;

function Panel() {
  return (
    <div>
      {/* Ця кнопка буде великою і синьою */}
      <ActionButton primary size="large">
        Зберегти
      </ActionButton>

      {/* Ця кнопка буде маленькою і сірою */}
```

```
      <ActionButton>Скасувати</ActionButton>
    </div>
  );
}
```

Це означає, що вам більше не потрібно вручну перемикати 10 різних класів типу `btn-primary`, `btn-large` за допомогою конкатенації рядків. Ви керуєте дизайном через дані (State/Props).

6. Utility-first CSS: Філософія Tailwind

Незважаючи на потужність `styled-components`, останні кілька років світ Frontend захопила зовсім інша філософія — **Tailwind CSS**.

Tailwind — це так званий *Utility-first* фреймворк. Замість того, щоб писати класи з назвами сутностей (як `.card`, `.button`) і набивати їх десятками властивостей, Tailwind пропонує тисячі заздалегідь написаних мікрокласів (utilities). Кожен мікроклас робить рівно ОДНУ річ.

Наприклад:

- Клас `flex` означає `display: flex;`
- Клас `p-4` означає `padding: 1rem;`
- Клас `text-center` означає `text-align: center;`
- Клас `bg-red-500` означає червоний фон певного відтінку.

Ми просто накидаємо ці класи прямо в HTML (JSX) як Лего:

```
// Кнопка з допомогою Tailwind:
<button className="bg-blue-500 hover:bg-blue-700 text-white font-bold py-2 px-4
rounded">
  Натисни мене
</button>
```

Переваги Tailwind:

1. **Ви більше ніколи не пишете власний CSS.** Ви не вигадуєте назви для класів (немає мук вибору між `.wrapper`, `.container`, `.inner-box`).
2. **Єдина дизайн-система.** Всі відступи і кольори жорстко обмежені палітрою Tailwind. Дизайн виглядає набагато професійніше і гармонійніше з коробки.
3. **Блискавична швидкість розробки.** Вам не треба бігати між двома файлами (`App.jsx` та `App.css`), весь дизайн відбувається на місці.

4. **Легковаговість.** Перед відправкою у браузер, компілятор Tailwind сканує ваш код і видаляє всі тисячі класів, які ви **не** використали. Фінальний CSS файл важить лічені кілобайти.

7. Базові класи Tailwind (Шпаргалка)

- **Розміри та відступи (Spacing):**
  - `p-` (padding), `m-` (margin).
  - Додаємо сторону: `pt-` (top), `pr-` (right), `px-` (ліво+право), `py-` (верх+низ).
  - Цифра в кінці — це множник (1 = 0.25rem/4px). Тобто `p-4` = 16px відступу з усіх боків.
- **Типографіка:**
  - `text-sm`, `text-lg`, `text-xl` (розмір шрифта).
  - `font-bold`, `font-semibold` (товщина).
- **Кольори:**
  - `bg-red-500` (фон червоний середньої насиченості), `text-white` (білий текст).
- **Flexbox та Grid:**
  - `flex`, `flex-col` (у стовпець), `justify-center`, `items-center` (центрування).
  - `grid`, `grid-cols-3` (сітка на 3 колонки).
- **Модифікатори станів:**
  - `hover:bg-red-600` (фон зміниться тільки при наведенні мишки).
  - `focus:border-blue-500` (зміна рамки при кліку в поле вводу).
- **Адаптивність (Mobile-first):**
  - Класи діють за замовчуванням на всі екрани (від мобільного).
  - Модифікатори `md:` (планшет) або `lg:` (десктоп) змінюють дизайн на великих екранах.
  - `<div className="w-full md:w-1/2">...</div>` (На мобілці ширина 100%, на екранах більше 768px — 50%).

8. Порівняння підходів: коли що обрати?

Немає єдиного "правильного" шляху стилізації в React. Вибір залежить від команди та проекту:

Інструмент	Плюси	Мінуси	Коли використовувати
CSS Modules	Нульовий поріг входу (звичайний	Немає прямого доступу до Props. Складно	Для маленьких проектів, або якщо команда тільки

	CSS), не потрібні нові бібліотеки. Гарантія ізоляції.	будувати загальні дизайн-системи.	переходить зі звичайного HTML/CSS.
Styled Components (CSS-in-JS)	Потужна динаміка (пропси керують CSS). Компоненти мають семантичні імена (`<Container>`, `<Header>`).	Уповільнює рендеринг складної сторінки (CSS генерується "на льоту" через JS).	Для створення UI-кітів (Бібліотек компонентів), де кожен елемент є інкапсульованою логічною одиницею з десятками станів.
Tailwind CSS	Нереальна швидкість верстки. Єдина система відступів/кольорів. Нульовий розмір фінального CSS-файлу.	JSX перетворюється на нечитабельне "болото" довгих класів (захаращення розмітки). Потрібно вчити багато нових абревіатур.	Для швидких стартапів, прототипів (MVP) та великих проектів, де потрібно багато кастомного і швидкого UI. Це індустріальний стандарт №1 сьогодні.

Висновки

1. Проблема глобального CSS змусила розробників шукати шляхи ізоляції стилів: спочатку через методології BEM, потім через модульний підхід.
2. Вбудованим у сучасні збірки (як Vite) механізмом безконфліктного імпорту стилів є CSS Modules.
3. CSS-in-JS (і його найяскравіший представник `styled-components`) пропонує створювати стилізовані HTML-теги як готові React-компоненти за допомогою Tagged Template Literals. Це дозволяє прокидати Props зі State прямо всередину CSS.
4. Tailwind CSS докорінно змінив парадигму: замість кастомних CSS-файлів розробники будують UI безпосередньо в JSX за допомогою заздалегідь підготовлених мікрокласів (Utility-first). Це прискорює розробку та гарантує консистентність дизайну.

Джерела

1. Styled Components Documentation
2. Tailwind CSS Official Site — неймовірно зручна документація-пошуковик по всіх класах.
3. The Evolution of CSS: From CSS to Styled Components — хороша стаття про еволюцію стилів.

Запитання для самоперевірки

1. У чому полягає найбільша небезпека (і біль) використання одного глобального файлу `style.css` у великих веб-додатках (до епохи ізоляції)? Що таке "Специфічність CSS"?
2. Поясніть принцип "CSS-in-JS" (на прикладі Styled Components). Що фізично імпортується і що ми пишемо у зворотному апострофі?
3. В чому головна супер-сила Styled components порівняно навіть з CSS Modules? Що компонент CSS-in JS може "спіймати" від своїх батьків для перетворення кольору/ширини динамічно?
4. Розшифруйте ідеологію "Utility-First" на якій побудовий Tailwind CSS? Як би ви описали клас `.card` і `.btn` у термінах мікро-класів (просторо/колір)?
5. Що означають класи Tailwind `m-4`, `p-2`, `bg-blue-600`, `hover:text-red-300`, `flex`?
6. З яким головним "Мінусом" Tailwind стикаються більшість розробників, коли переходять на нього (чисто візуальна естетика вашого коду)?
7. Яким чином Tailwind гарантує, що незважаючи на тисячі доступних класів-утиліт, фінальний CSS-файл для браузера буде "важити" лічені кілобайти?